

Web Applications

Event-Driven Architectures in JavaScript

Suheil Hammoud

What is Event-Driven Architecture?

- A design paradigm where the flow is driven by events.
- Components communicate by emitting and reacting to events.
- Promotes **decoupling**, **asynchronous behavior**, and **scalability**.

Benefits of Event-Driven Architecture

- **Loose coupling:** Components don't directly depend on each other.
- **Asynchronous:** Improves responsiveness and performance.
- **Modular:** Easier to test, debug, and maintain.
- **Flexible:** Easy to add or remove listeners.

Event Loop Review

- JS uses a **single-threaded event loop**.
- Queue system: call stack, task queue, microtask queue.
- `setTimeout`, `Promise`, `async/await`, and DOM events use this loop.

 The event loop is the backbone of EDA in JS.

Node.js: EventEmitter Class

ES6+ support allows clean, class-based EDA in Node.js.

```
import { EventEmitter } from 'events';

class MyEmitter extends EventEmitter {}

const emitter = new MyEmitter();

emitter.on('data', (msg) => {
  console.log(`Received: ${msg}`);
});

emitter.emit('data', 'Hello, event-driven world!');
```

✓ Works in Node.js with "type": "module" or .mjs.

Node.js EventEmitter Methods

- `.on(event, listener)`
- `.once(event, listener)`
- `.off(event, listener)` (Node 10+)
- `.emit(event, [args])`
- `.removeAllListeners(event)`

Useful for CLI apps, file watchers, logging systems, microservices.

Browser: Custom Events with EventTarget

```
class MyComponent extends EventTarget {  
  trigger() {  
    this.dispatchEvent(new Event('updated'));  
  }  
}  
  
const comp = new MyComponent();  
  
comp.addEventListener('updated', () => {  
  console.log('Component updated!');},  
  // { once: true } 👉 Listener is automatically removed when the component is destroyed.  
);  
  
comp.trigger();
```

✅ Works natively in browsers, excellent for UI interactions.

Building a Global Event Bus with ES Modules

```
// eventBus.js
export const eventBus = new EventTarget();

// ui.js
import { eventBus } from './eventBus.js';

eventBus.addEventListener('login', () => console.log('User logged in'));

// auth.js
import { eventBus } from './eventBus.js';

eventBus.dispatchEvent(new Event('login'));
```

 Modules allow decoupled communication between components.

Asynchronous Event Handling with Promises

```
emitter.on('fetch', async () => {  
  const res = await fetch('/api/data');  
  const json = await res.json();  
  console.log(json);  
});
```

 Combine EDA with `async/await` for non-blocking workflows.

Convert Events to Promises using `once`

```
import { once } from 'events';

async function waitForEvent(emitter, event) {
  const [result] = await once(emitter, event);
  console.log(`Got event data: ${result}`);
}
```

⚡ Makes event-handling compatible with `Promise` pipelines.

Custom Lightweight Event Bus (Class-based)

```
class EventBus {  
  constructor() {  
    this.listeners = new Map();  
  }  
  
  on(event, cb) {  
    if (!this.listeners.has(event)) this.listeners.set(event, []);  
    this.listeners.get(event).push(cb);  
  }  
  
  emit(event, data) {  
    (this.listeners.get(event) || []).forEach(cb => cb(data));  
  }  
  
  off(event, cb) {  
    this.listeners.set(event, this.listeners.get(event).filter(fn => fn !== cb));  
  }  
}
```

 A minimal implementation for apps without external libraries.

Frontend EDA with RxJS (Reactive Streams)

```
import { fromEvent } from 'rxjs';

fromEvent(document, 'click')
  .subscribe(() => console.log('Document clicked!'));
```

- Reactive programming with Observables
- Advanced operators: `debounceTime`, `mergeMap`, `switchMap`

💡 Powerful for real-time apps, analytics, auto-complete, etc.

Real-World Use Cases

- UI interactions (forms, buttons, navigation)
- Game loops (animation, physics, input)
- WebSockets and real-time chat
- Microservice communication (Kafka, Redis pub/sub)
- Logging and monitoring systems

Pitfalls & Best Practices

Common Mistakes:

- Forgetting to remove listeners
- Emitting too frequently (performance hit)
- Overcomplicating with too many events

Best Practices:

- Always clean up with `.off()` / `removeEventListener`
- Log event flows for debugging
- Use events to decouple, not hide control flow

Summary

- JavaScript's event-driven model is fundamental and powerful.
- ES6+ features like classes, modules, and async/await enhance EDA.
- Useful across both frontend and backend ecosystems.
- Mastering EDA leads to cleaner, more scalable JS applications.

 **Key takeaway:** Emit events to decouple logic, enhance modularity, and scale effectively.